Step 1 — Collect Training Data (Manual Driving)

I began by manually driving the car in the Udacity simulator.
During each lap, the simulator recorded:

- center camera images
- steering angle
- throttle
- speed

All of this was stored in:

- `driving_log.csv`
- `IMG/` folder with the images

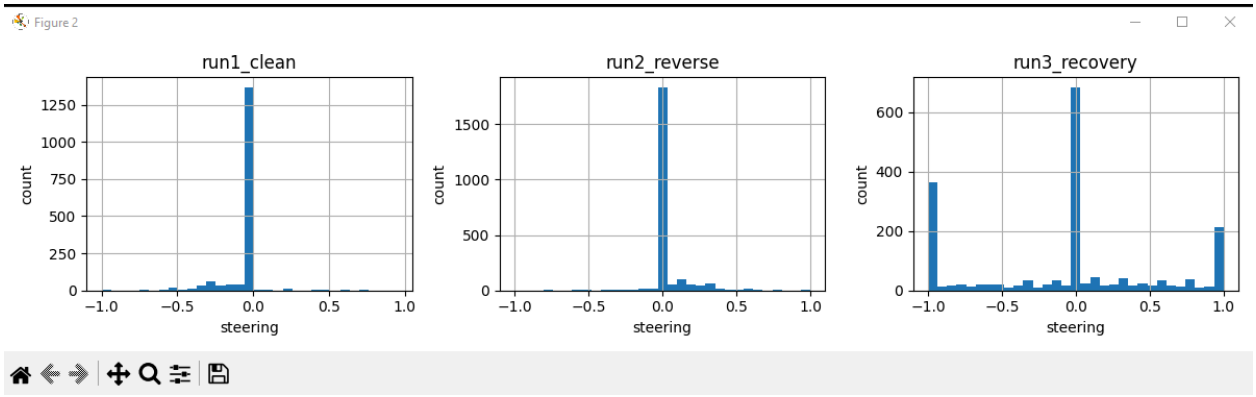
We repeated this process several times to get:

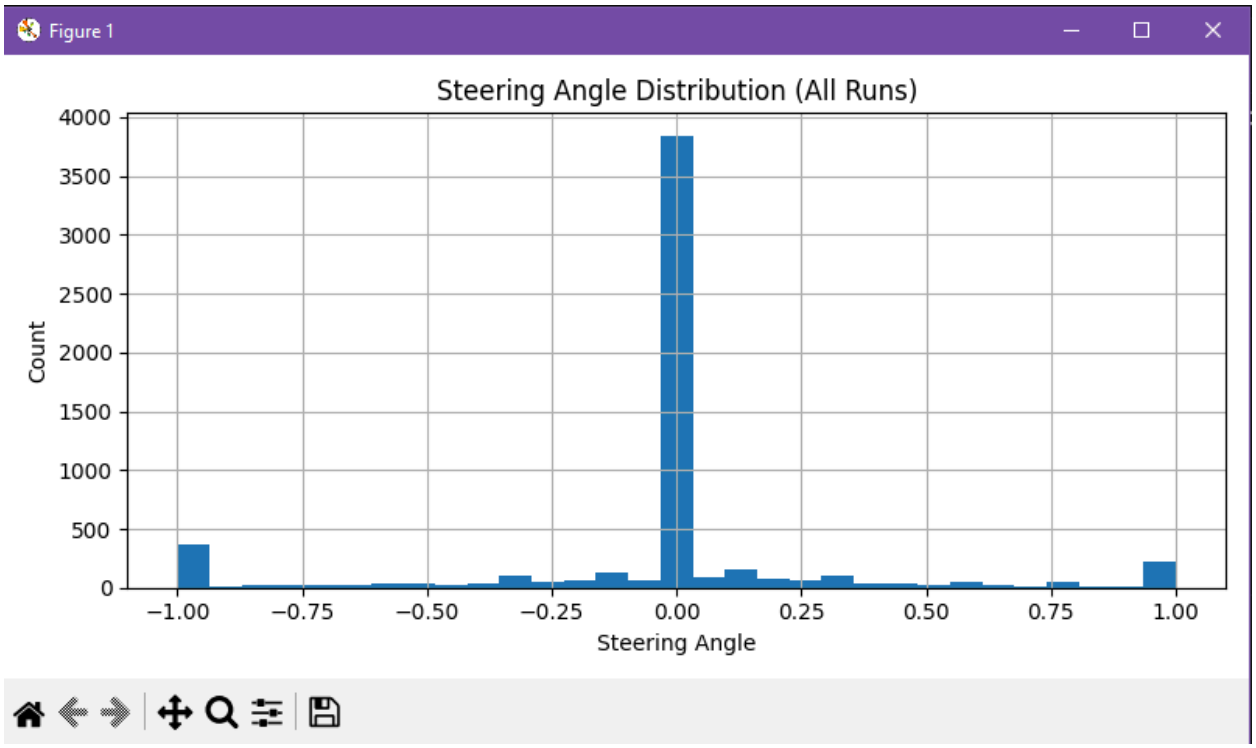
- cleaner laps
- smoother steering
- reduced jerky movements
- different tracks for better generalization

Goal: give the model real human examples of lane-keeping.

Step 2 — Explore & Analyze the Raw Data

Before training, I inspected the dataset to understand the distribution.



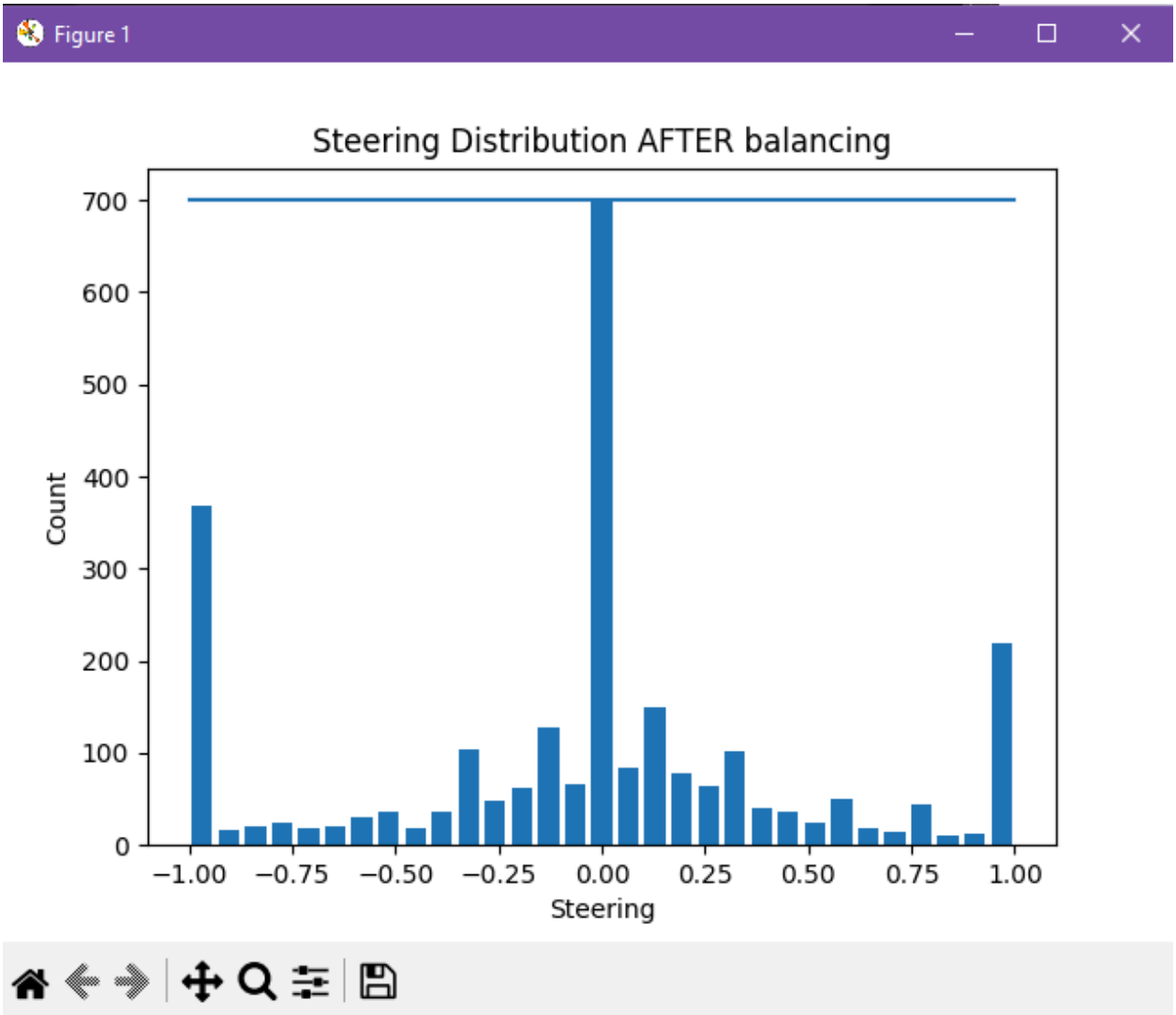


Overall what we found with the raw data was:

- Straight driving dominates, model will learn to always predict 0.0
- Recovery data only made up only 32%. These runs are important since they display how to correct drifting off center and things like re-entering the center
- Reverse laps improve generalization but straight driving still dominates
- Run1_clean(2280) + run2_reverse(1850) + run3_recovery(1662) = 5792

Step 3 — Downsample “Straight Driving” Frames & Cleaning up Raw data

After reviewing the issues with the data we managed to remove redundant 0.0 images by alot



- 0.0 spike is capped at ~700 samples
- Steering angles now have a wide spread across -1.00 to +1.00
- Extreme ±1.0 turns are still present for strong corrections
- More natural looking distribution
- Run1_clean(1275) + run2_reverse(800) + run3_recovery(574) = 2649

Step 4 — Split For Training and Validation

Before preprocessing the images, we first convert the cleaned dataframe into two arrays:

1. one containing the full file paths of every image
2. one containing the matching steering angles.

This step organizes the dataset into a format the model can easily consume later during training.

Once the images and labels are extracted, we split them into a training set and a validation set using an 80/20 ratio.

The training set is used to teach the model, while the validation set is kept separate to measure how well the model generalizes to unseen data. This split ensures the model isn't just memorizing the training images and allows us to monitor overfitting during training.

- Total images (after balancing): **2649**
- Train: **2119**
- Val: **530**

Step 5 — Image Preprocessing and Data Augmentation

We implemented functions that:

- **preprocess_image()**: turns **raw images** to clean, standardized **road only images** for the model
- **random_brightness():** simulates different lighting conditions
- **random_translate():** simulates the car drifting inside the lane, helps model learn recovery behavior
- **random_flip():** flipping doubles our data & makes left/right behaviour more symmetric. Mitigates left or right turn bias
- **augment_image():** randomly applies above functions to increase effectiveness and diversity of the data for better learning
- zoom_image():
- **batch_generator():**
    - shuffles sample order
    - Takes a chunk of paths and loads the images, converting BGR to RGB and applies augmentation
    - Lets us train large datasets efficiently, with fresh random augmentations every epoch

---

Step 6 — Creating the Model

We implement the NVIDIA End-to-End Deep Learning model for the model. It comes with 5 convolutional layer:

1. **First Conv Layer:** `Conv2D(24,(5,5),strides=(2,2))`**:**
    - **24 filters** detect basic features like edges, lines, and gradients
    - **5x5 kernel** capture local patterns in small region
    - **Stride of 2** downsamples the image, making it more compute efficient
    - *Learns where the edges are in the image*
2. **Second Conv Layer:** `Conv2D(36,(5,5),strides=(2,2))`
    - **36 filters** detect more complex combinations of edges
    - **5x5 kernel** stays the same
    - **Stride of 2** downsamples the image, making it more compute efficient
    - *Learns how edges combine to form shapes like lane lines, curves or road boundaries*
3. **Third Conv Layer:** `Conv2D(48,(5,5),strides=(2,2))`
    - **48 filters** learn even more abstract patterns
    - Final layer with stride=2 **downsampling**
    - **Stride of 2** downsamples the image, making it more compute efficient
    - *Learns What patterns these shapes form*
4. **Fourth Conv Layer:** `Conv2D(64,(3,3))`
    - **64 filters** for high-level feature detection
    - **3×3 kernel** for fine-grained details after downsampling
    - *Learns specific road structures, Lane positions and road curvature characteristics*
5. **Fifth Conv Layer:** `Conv2D(64,(3,3))`
    - **64 filters** to refine high-level features
    - **3×3 kernel** for fine-grained details after downsampling
    - *Learns Final refinement of road understanding before making a decision*

Each layer needs **MORE filters** to capture **MORE complex combinations** of the simpler features from the previous layer. Start with **larger kernels** and **move to smaller ones**. Strides to **reduce computation**

- **Flatten Layer**: Converts 2D images to 1D vectors and pass it off to dense layers
- **Dense Layer:**
    - **Dense(100):** Takes all extracted features and starts learning "what combination of road features → what steering angle?"
    - **Dense(50):** Further compresses and refines the decision-making
    - **Dense(10):** Final compression before output
    - **Dense(1):** The output neuron - a single number representing the steering angle

Elu (Exponential Linear Unit) is used instead of reLu as it allows negative values, and NVIDIA found it worked better for continuous control tasks like steering

MSE is used since this is a regression problem. MSE penalizes large mistakes greatly, leading to the model keeping a tight steering angle

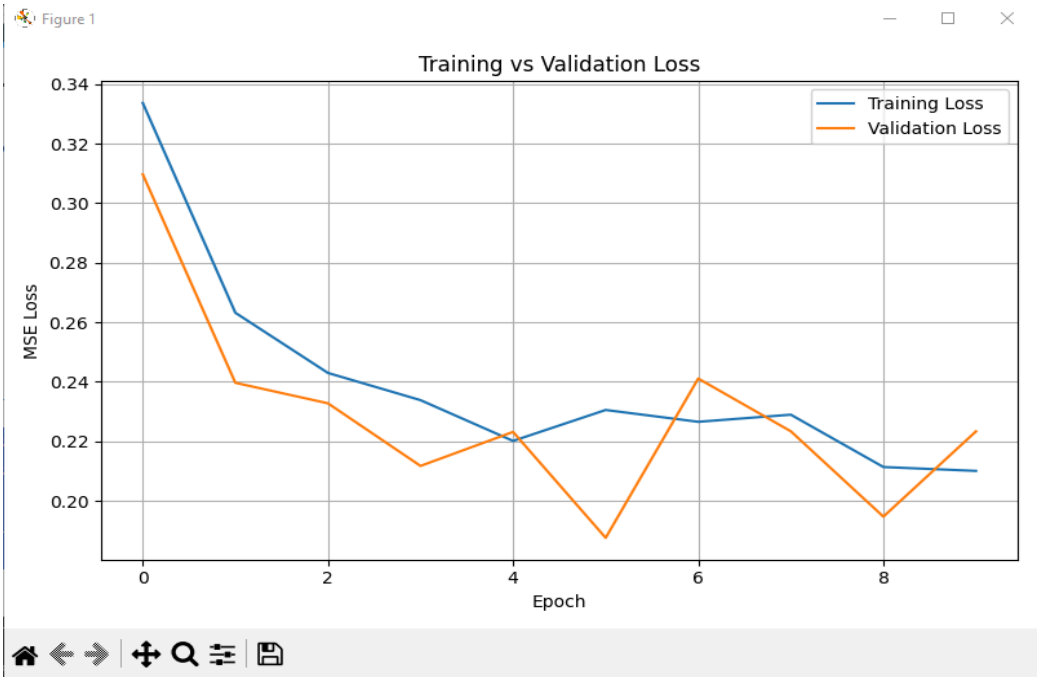
Adam() is the most widely used optimizer in DL as it adjusts learning rate for each weight & works well with noisy real world data

---

Step 7 — Training the model

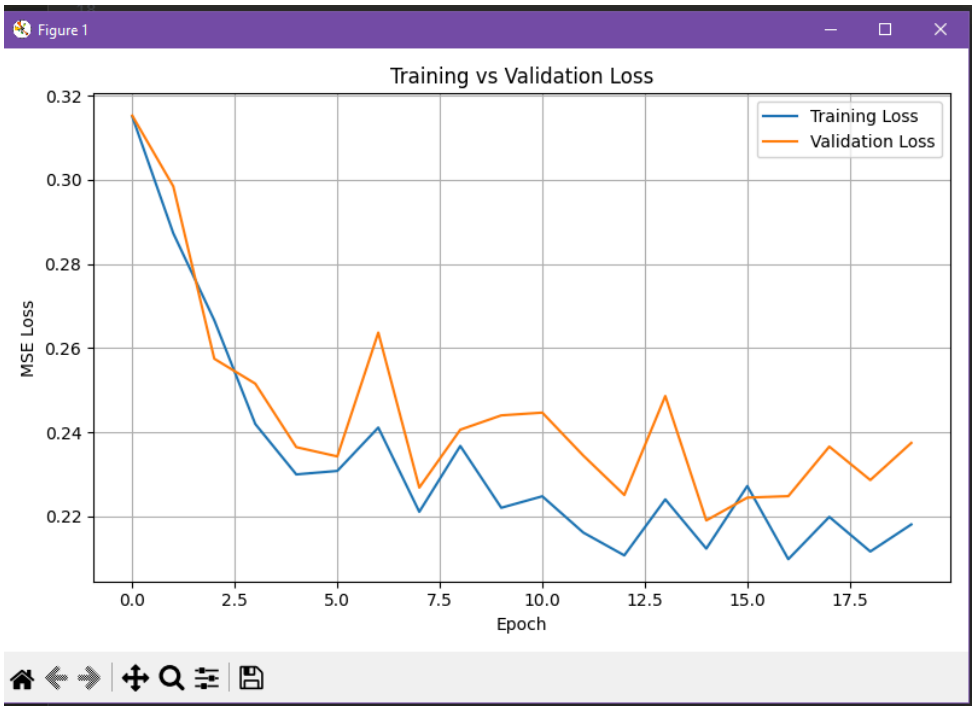
Training loss measures our model's performance on the **data it's learning from**, while **validation loss** measures its ability to generalize to **new, unseen data**. After 10 epochs, **loss** = 0.2114 and **val_loss** = 0.2233.

- As of right now, the training and validation loss is close, which means our augmentation and balancing **worked**,
- validation is following the training data loosely. Overall, our model is not overfitting, but it is still **underperforming…**
- **Loss is too high**, model wont be able to steer smoothly and will not be able to follow curves
- The **validation curve is too noisy**, its fluctuating in its guesses too much
- Our model is **underfitting.**



To improve the performance, we will have to…

- Train longer (**20–30 epochs**)
- Use a **smaller learning rate**
- Added callbacks:
    - EarlyStopping: Stops training when improvement plateaus
    - ReduceLROnPlateau: adjusts the learning rate if progress stalls, tunes it to make smaller, more careful weight updates. *Like lowering the temperature on the stove when you're cooking*



Step 8 — Build the Real-Time Inference Server

Real-Time Inference Pipeline

- Built using Flask and Socket.IO for real-time communication
- Simulator streams camera images to Python script
- Images decoded and preprocessed using training pipeline
- Processed frames passed through NVIDIA-model for steering prediction
- Steering stabilization via bias, scaling, smoothing, and throttle control
- Predicted steering and throttle sent back to simulator each frame
- Enables autonomous track navigation

python scripts/test_simulation.py --model models/nvidia_model.h5

python scripts/test_simulation.py --model models/nvidia_model.h5 --max-speed 7 --steering-bias 0.0 --steering-scale 1.0 --smooth 0.0

python scripts/test_simulation.py --model models/nvidia_model.h5 --max-speed 7 --steering-scale 1.1 --steering-bias 0.0 --smooth 0.3

Demo Test Run here:

https://youtube.com/shorts/lkaNqqNlJ9s?feature=share

Step 9 — Room for Improvement

The model could follow most of the track, including gentle and moderate curves, but consistently failed on one sharp left-hand bend. This could be due to the fact that the model is trained on MSE over all frames, and mostly frames are 0.0, so sharp turns are rare and risky, in MSE terms.

To address this permanently, we would need to:

- Collect more targeted data
- Refine data balancing: downsample 0.00 even more
